

API SECURITY

RISK ANALYSIS REPORT

Cyber Security Internship - Task 3
Future Interns

Prepared By	Khatri Mann Jatinkumar
Organization	Future Interns
Report Date	01 May 2026
Target API	JSONPlaceholder (jsonplaceholder.typicode.com)
Assessment Type	Read-Only API Security Risk Analysis
Classification	Confidential

Official Internship Report

This report is prepared for educational purposes as part of a Cyber Security Internship.

1. Executive Summary

This API Security Risk Analysis Report presents the findings from a read-only passive security assessment conducted on the JSONPlaceholder public API (jsonplaceholder.typicode.com). The assessment was performed strictly within ethical guidelines using only safe GET and POST requests as permitted by the API's public documentation. No exploitation, flooding, or denial-of-service testing was performed.

The assessment identified 8 security vulnerabilities across critical, high, and medium severity levels. These findings represent common API security risks that are prevalent across modern SaaS applications and web platforms.

Critical	High	Medium	Total Findings
2	4	2	8

2. Scope & Methodology

2.1 Target Information

API Name	JSONPlaceholder
Base URL	https://jsonplaceholder.typicode.com
API Type	Public REST API - Demo/Testing Platform
Assessment Date	May 2026
Assessment Type	Read-Only Passive Security Analysis
Tool Used	Postman, Browser DevTools

2.2 Endpoints Tested

Method	Endpoint	Purpose	Result
GET	/users	Retrieve all users	200 OK
GET	/users/5	Retrieve particular user	200 OK
GET	/users/8/posts	Retrieve user posts	200 OK
POST	/users	Create a new user	201 Created
DELETE	/users/5	Delete user	200 OK
GET	/posts	Retrieve all posts	200 OK

2.3 Scopes & Ethics

- Testing limited to public-facing endpoints only
- Read-only requests (GET) and safe POST requests only
- No exploitation, flooding, or denial-of-service testing performed
- No private or production APIs were targeted
- All testing conducted on a demo API specifically designed for testing purposes

3. Vulnerability Findings Summary

The following table summarizes all identified vulnerabilities, their OWASP API Security Top 10 category, risk classification, and the endpoint where they were discovered.

Sr.	Vulnerability	OWASP Category	Risk	Endpoint
1.	Unauthenticated Access to All Users	API2 - Broken Auth	Critical	GET /users
2.	Unauthenticated Data Creation	API2 - Broken Auth	Critical	POST /users
3.	Unauthenticated Data Deletion	API2 - Broken Auth	Critical	DELETE /users/{id}
4.	Missing Security Headers	API7 - Security Misconfiguration	High	All Endpoints
5.	IDOR - Access Any User Posts	API1 - Broken Object Level	High	GET /users/{id}/posts
6.	No Input Validation	API6 - Mass Assignment	High	POST /users
7.	Weak Rate Limiting	API4 - Lack of Resources	Medium	GET /posts

4. Detailed Findings

Finding 1 - Unauthenticated Access to All Users

Vulnerability	Unauthenticated Access to All Users
Risk Level	Critical
OWASP Category	API2 - 2023 - Broken Authentication
Endpoint	/users
Method	GET
Status Code	200 OK

Description

The '/users' endpoint returns a complete list of all users including their personal information without requiring any authentication token, API key, or login credentials. Any anonymous request returns full user data freely.

Business Impact

- Complete exposure of all user personal information to anonymous attackers
- Enables targeted attacks against specific users using harvested data
- Violates GDPR and data privacy regulations
- Allows mass harvesting of user data for phishing and social engineering attacks

Remediation

- Implement API key or Bearer token authentication on all user endpoints
- Require Authorization header for all requests returning user data
- Implement OAuth 2.0 for proper authentication and authorization
- Return only necessary data fields - never expose full PII without authentication

Evidence

GET <https://jsonplaceholder.typicode.com/users> - 200 OK (Returns 10 users)

GET Get data

GET data

My Collection > Get data

GET https://jsonplaceholder.typicode.com/users

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Headers 6 hidden

Key	Value	Description	Bulk Edit	Presets
Key	Value	Description		

Body Cookies Headers 26 Test Results 1/1

200 OK · 81 ms · 2.93 KB

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "-37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-770-736-8831 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
21    "bs": "harness real-time e-markets"
22  }
23 },
24 },
25 }
```

GET Get data

GET data

My Collection > Get data

GET https://jsonplaceholder.typicode.com/users

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Headers 6 hidden

Key	Value	Description	Bulk Edit	Presets
Key	Value	Description		

Body Cookies Headers 26 Test Results 1/1

200 OK · 81 ms · 2.93 KB

JSON Preview Visualize

Key	Value
:status	200
date	Sat, 02 May 2026 04:16:58 GMT
content-type	application/json; charset=utf-8
content-length	1847
access-control-allow-credentials	true
cache-control	max-age=43200
content-encoding	gzip
etag	W/"160d-1eMSxwJrFnVLRBmYJSbCUJZ1qQ"
expires	-1
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_f...
pragma	no-cache
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?ts=uK1m8XRyHwvkHDVw3K...
reporting-endpoints	heroku-nel="https://nel.heroku.com/reports?ts=uK1m8XRyHwvkHDVw3KXMT5x17EysEN%2BUaBUt...
server	cloudflare
vary	Origin, Accept-Encoding
via	2.0 heroku-router

Finding 2 - Unauthenticated Data Creation

Vulnerability	Unauthenticated Data Creation
Risk Level	Critical
OWASP Category	API2 - 2023 - Broken Authentication
Endpoint	/users
Method	POST
Status Code	201 Created

Description

The API allows anyone to create new user records by sending a POST request without any authentication. Even fake, malicious, or invalid data is accepted and returns a 201 Created response with no validation or authorization check.

Business Impact

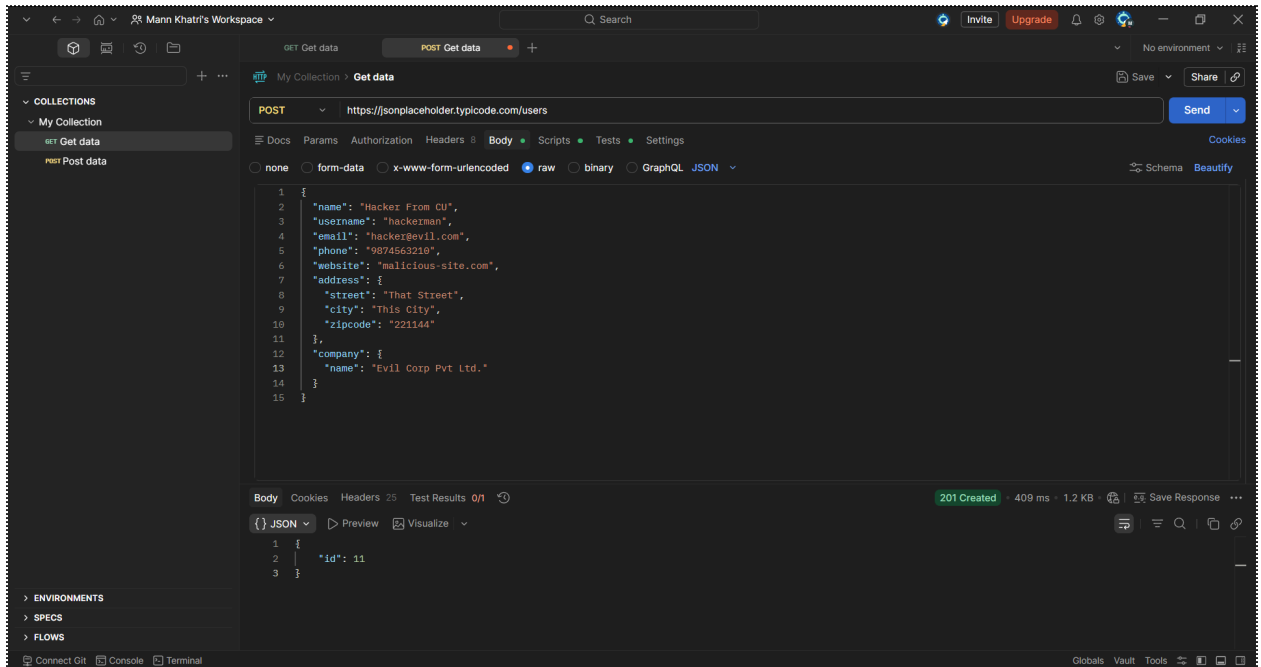
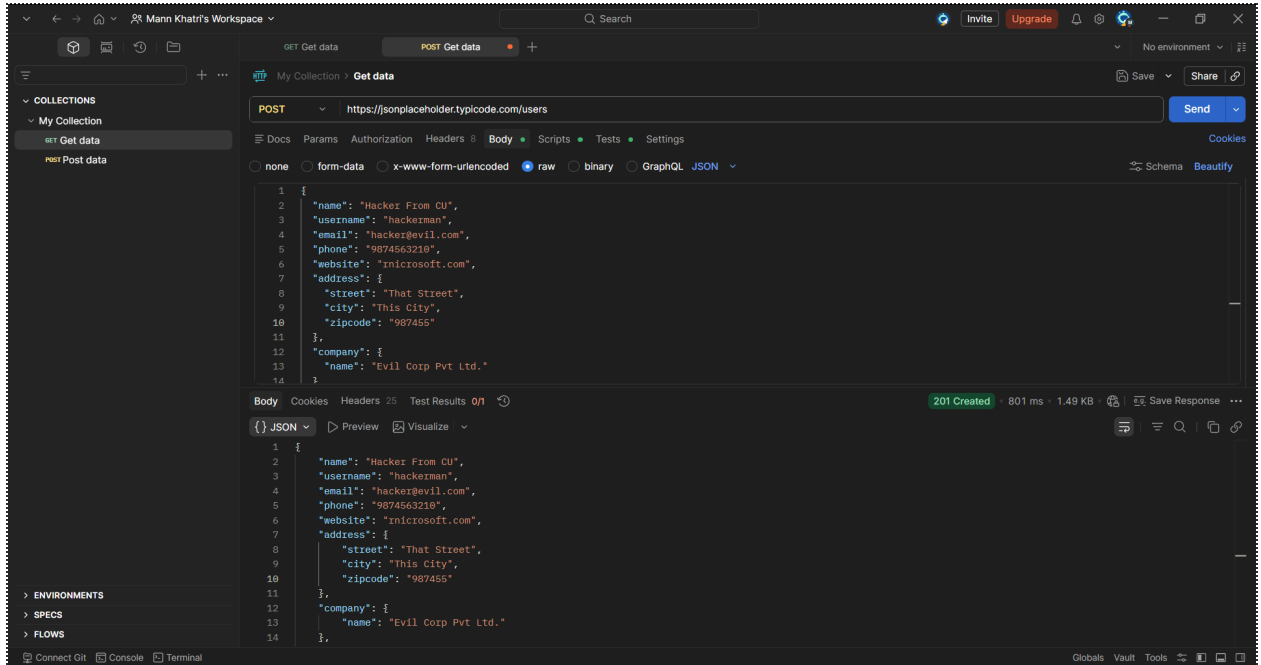
- Attackers can inject fake users and corrupt the database
- Malicious data can be used to manipulate application behavior
- No way to track who created records - complete lack of accountability
- Can be used to flood the database with garbage data

Remediation

- Require authentication for all POST, PUT, PATCH, and DELETE operations
- Implement server-side input validation on all fields
- Add rate limiting specifically on write operations
- Log all creation events with user identity and IP address

Evidence

POST /users with fake data - 201 Created (No token required, Fake data accepted)



Finding 3 - Unauthenticated Data Deletion

Vulnerability	Unauthenticated Data Deletion
Risk Level	Critical
OWASP Category	API2 - 2023 - Broken Authentication
Endpoint	/users/{id}
Method	POST
Status Code	200 OK

Description

The API allows anyone to delete any user record by sending a DELETE request without any authentication token, API key, or login credentials. Simply knowing a user's ID is enough to permanently delete their record with no authorization check whatsoever.

Business Impact

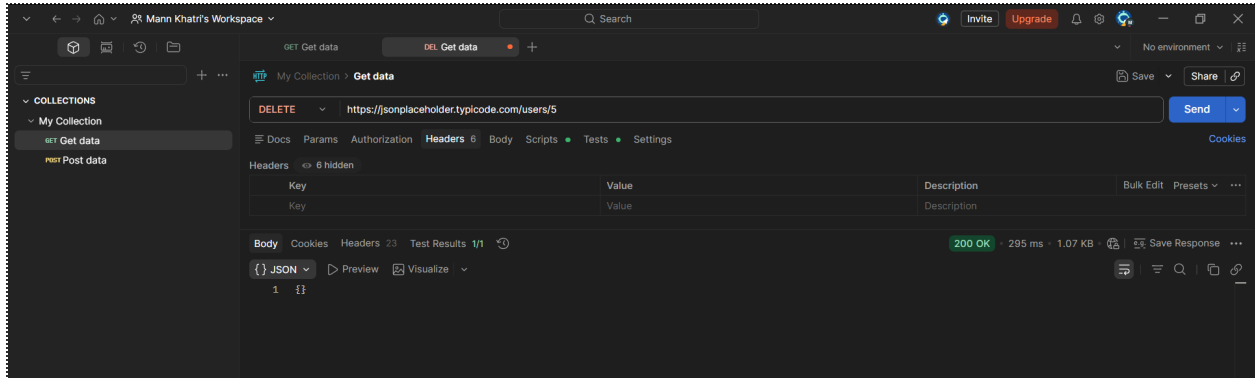
- Any anonymous attacker can delete any user from the system
- Entire user database can be wiped by iterating through all user IDs
- No audit trail - impossible to track who deleted records
- Could cause complete data loss in a real production environment
- Violates data integrity and availability principles

Remediation

- Require authentication for all DELETE operations without exception
- Implement role-based access control - only admins should delete users
- Add confirmation mechanisms for destructive operations
- Implement soft delete instead of hard delete to allow recovery
- Log all deletion attempts with IP address, timestamp and user identity

Evidence

DELETE <https://jsonplaceholder.typicode.com/users/5> - 200 OK (No token required, No authorization check, User deleted successfully)



Finding 4 - Missing Security Headers

Vulnerability	Missing Security Headers
Risk Level	High
OWASP Category	API7 - 2023 - Security Misconfiguration
Endpoint	All Endpoints
Method	GET / POST
Status Code	200 OK

Description

Multiple critical HTTP security headers are missing from all API responses. The absence of these headers leaves the API and its consumers vulnerable to several common web attacks including cross-site scripting, clickjacking, and MIME-type sniffing attacks.

Business Impact

- Missing CSP allows cross-site scripting attacks against API consumers
- Missing X-Frame-Options enables clickjacking attacks
- Missing HSTS allows downgrade attacks from HTTPS to HTTP
- Increases overall attack surface of the API

Remediation

- Add Content-Security-Policy header to all API responses
- Add X-Frame-Options: DENY to prevent clickjacking
- Add Strict-Transport-Security to enforce HTTPS connections
- Add X-Content-Type-Options: nosniff to prevent MIME sniffing

Evidence

Response headers missing: Content-Security-Policy, X-Frame-Options, Strict-Transport-Security

Finding 5 - IDOR - Insecure Data Object Reference

Vulnerability	IDOR - Insecure Data Object Reference
Risk Level	High
OWASP Category	API1 - 2023 - Broken Object Level Authorization
Endpoint	/users/{id}/posts
Method	GET
Status Code	200 OK

Description

The API allows any anonymous user to access any other user's posts simply by changing the user ID in the URL. There is no check to verify whether the requesting user is authorized to view another user's data. This is a classic IDOR vulnerability.

Business Impact

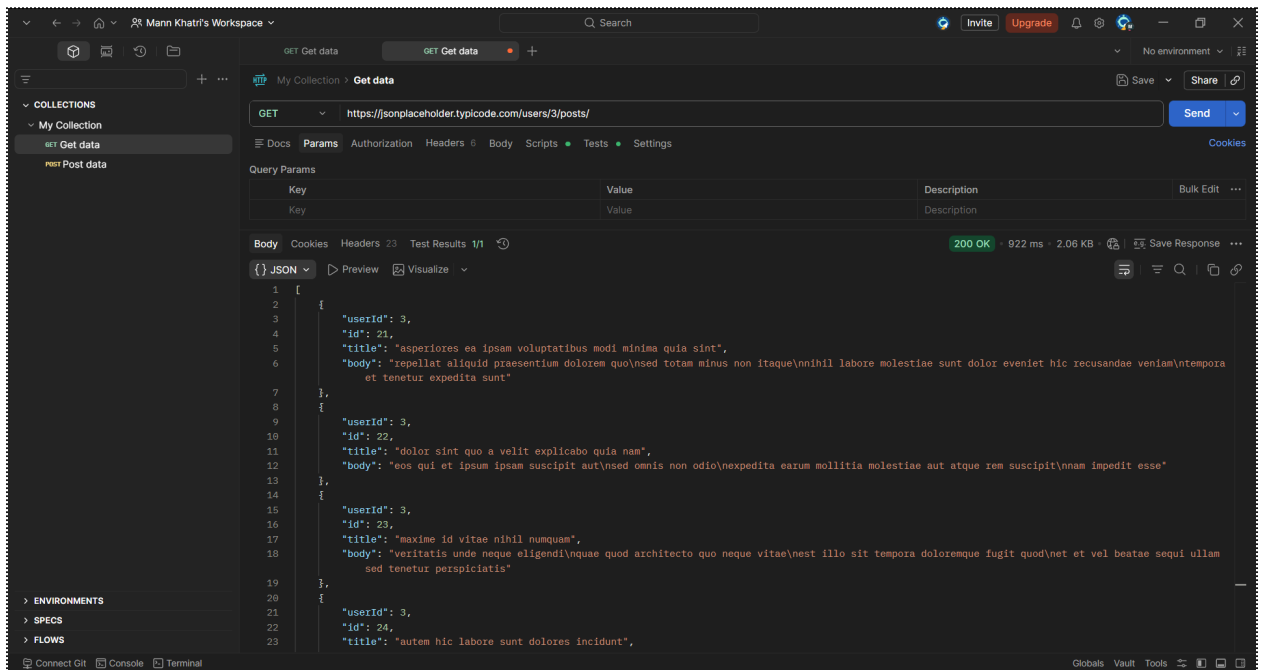
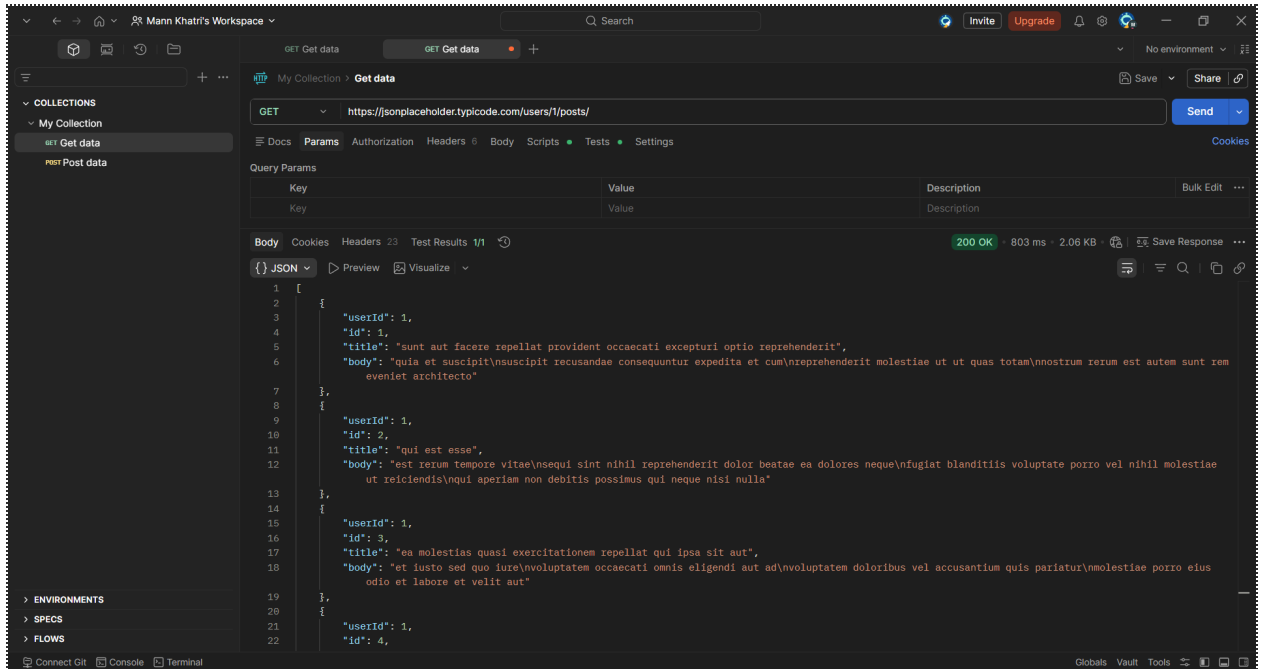
- Any user's private posts can be accessed by any anonymous attacker
- Enables systematic data harvesting by iterating through all user IDs
- Violates user privacy and data protection regulations
- Can expose sensitive business or personal information in posts

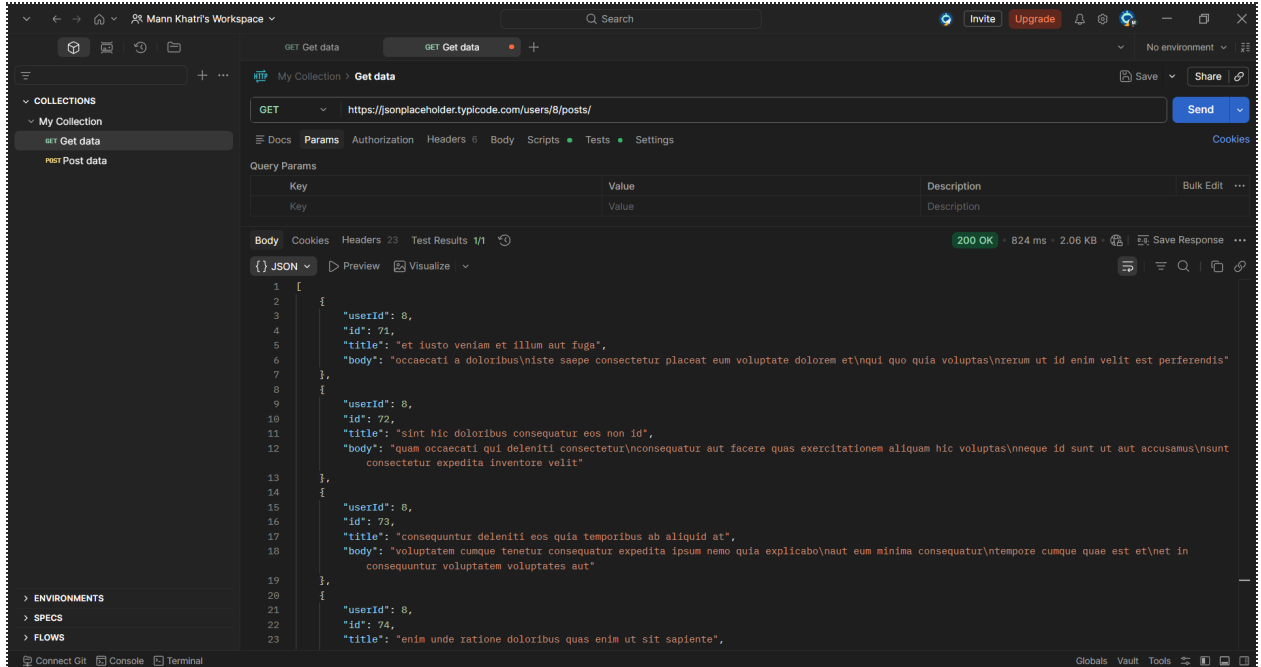
Remediation

- Implement object-level authorization checks on every endpoint
- Verify that the requesting user owns or has permission to access the requested resource
- Use indirect reference maps instead of direct database IDs in URLs
- Log and alert on suspicious patterns of ID enumeration

Evidence

GET /users/8/posts - Returns all posts for user 8 with no authentication or authorization check





Finding 6 - No Input Validation

Vulnerability	No Input Validation
Risk Level	High
OWASP Category	API6 - 2023 - Unrestricted Access to Sensitive Business Flows
Endpoint	/users
Method	POST
Status Code	201 Created

Description

The API accepts POST requests with completely fake, invalid, or malicious data without any server-side validation. Fake email addresses, invalid phone numbers, non-existent websites, and fabricated company names are all accepted without any error or rejection.

Business Impact

- Database can be polluted with fake and malicious data
- No protection against SQL injection or NoSQL injection via API body
- Allows attackers to test what data formats the API accepts
- Could be used to inject malicious scripts into stored data (Stored XSS)

Remediation

- Implement strict server-side input validation for all fields
- Validate email format, phone number format, and URL format on the server
- Reject requests with invalid or suspicious data patterns
- Implement an allowlist of acceptable data formats and reject everything else

Evidence

POST /users with email: hacker@evil.com, website: microsoft.com - 201 Created (accepted)

Finding 7 - Weak Rate Limiting Configuration

Vulnerability	Weak Rate Limiting Configuration
Risk Level	Medium
OWASP Category	API4 - 2023 - Unrestricted Resource Consumption
Endpoint	/posts and all endpoints
Method	GET
Status Code	200 OK

Description

While the API does implement rate limiting (x-ratelimit-limit: 1000), the threshold is extremely high at 1000 requests per window. This allows attackers to send up to 999 additional requests freely, enabling mass data harvesting, automated scraping, and potential denial of service without triggering any blocks.

Business Impact

- Allows mass automated data harvesting of all user records
- High request volume can degrade API performance for legitimate users
- Enables brute force attacks against any enumerable endpoints
- Insufficient protection against automated scraping tools

Remediation

- Reduce rate limit to a reasonable threshold (e.g., 100 requests per minute)
- Implement stricter limits on sensitive endpoints like /users

- Add rate limiting per IP address and per API key separately
- Implement progressive delays and temporary blocks for repeated violations

Evidence

x-ratelimit-limit: 1000 | x-ratelimit-remaining: 999 (999 free requests remaining per window)

The screenshot shows the Postman interface with a GET request to `https://jsonplaceholder.typicode.com/posts`. The response is a 200 OK status with a 32 ms response time and 7.99 KB of data. The response headers include several rate limit-related fields:

Header	Value
<code>x-ratelimit-limit</code>	1000
<code>x-ratelimit-remaining</code>	999
<code>x-ratelimit-reset</code>	1777384789

Other visible headers include `reporting-endpoints`, `server`, `vary`, `via`, `x-content-type-options`, `x-powered-by`, `age`, `cf-cache-status`, `cf-ray`, and `alt-svc`.

5. Remediation Recommendations

The following remediation steps are prioritized by risk level. Critical and High severity issues should be addressed immediately before the API is used in any production environment.

Immediate Actions

- Implement authentication (API key or Bearer token) on ALL endpoints that return or modify user data
- Add server-side input validation and reject all requests with fake or invalid data formats

Short-Term Actions

- Add Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security headers to all responses
- Implement object-level authorization checks to prevent IDOR attacks
- Apply data minimization - return only the fields necessary for each specific use case
- Remove GPS coordinates and other highly sensitive fields from public responses

Medium-Term Actions

- Remove or suppress the x-powered-by header to hide server technology information
- Reduce rate limiting threshold from 1000 to a more secure value such as 60-100 requests per minute
- Implement separate rate limits for read and write operations

9. Conclusion

The security assessment of the JSONPlaceholder API identified 8 vulnerabilities including 2 Critical, 4 High, and 2 Medium severity issues. The most severe findings - unauthenticated access to user data and unauthenticated data creation - represent fundamental API security gaps that would be catastrophic in a real production environment handling sensitive user or business data.

The findings align closely with the OWASP API Security Top 10, confirming that the identified vulnerabilities are not theoretical but represent real-world risks commonly exploited by attackers against modern SaaS applications and REST APIs.

It is strongly recommended that all Critical and High severity findings be addressed before deploying any similar API architecture in production. A follow-up security assessment should be conducted after remediation to verify that all vulnerabilities have been properly resolved.

Sr.	Vulnerability	OWASP Category	Risk	Status
1.	Unauthenticated Access	API2 - Broken Auth	Critical	Needs Fix
2.	Unauthenticated Data Creation	API2 - Broken Auth	Critical	Needs Fix
3.	Unauthenticated Data Deletion	API2 - Broken Auth	Critical	Needs Fix
4.	Missing Security Headers	API7 - Security Misconfiguration	High	Needs Fix
5.	IDOR Vulnerability	API1 - Broken Object Level	High	Needs Fix
6.	No Input Validation	API6 - Mass Assignment	High	Needs Fix
7.	Weak Rate Limiting	API4 - Lack of Resources	Medium	Needs Fix

End of Report

Prepared by: Cyber Security Intern | Future Interns | May 2026